

Advanced Quantum Coding Project

Crest Koemel

February 2026

1 Theory

Spectrophotometers rely on the detection of photons on a detector. With the advent of multichannel detectors varying wavelengths of photons could be simultaneously measured. Separating the wavelength components of a beam of light spatially allows different wavelengths to hit different pixels. Assuming a 1D array means if we have a ten pixel detector, and a beam of light that is spatially spread out (according to wavelength) along the axis of the detector then we will have ten data points. Each of the pixel values spanning some range of wavelengths.

Methods for spatially separating the wavelength components a beam of light include diffraction gratings and prisms. The latter is described by refractivity which is nonlinear according to wavelength. The former is described by the grating equation:

$$\theta = \sin^{-1} \left(\frac{\lambda_{nm}}{\frac{1}{g}} \right)$$

Where g is the groove density or number of grooves per unit length on the diffraction grating; λ is the wavelength of light; and θ is the angle that the wavelength of light disperses.

Position on the detector can be calculated using the equation:

$$P = L \cdot \tan(\theta)$$

Where P is the position on the detector; L is the length from the grating; and θ is the angle of dispersion. Since the change of position per change in wavelength is relatively constant ($\frac{\Delta P}{\Delta \lambda} \approx \text{constant}$) between wavelengths of 300 nm and 1000 nm the spatial dispersion is treated as linear with respect to wavelength.

With a material that absorbs at known wavelengths it is possible to relate pixel to wavelength. Measuring the spectrum of a material gives absorbance values vs pixels. Picking peaks with known wavelengths allows for the assignment of pixels to wavelength. A linear fit between all combination of pixels to peaks allows for the interpolation of pixels which might reside between peaks.

With the pixel position changing as $\tan(\theta)$. We can also approximate the curvature as a cubic polynomial, in cases of large θ .

2 User Manual

This Program is made to aid in the calibration of a spectrograph utilizing a standard with known wavelengths of absorbance. The program takes a experimental spectrum of your standard as an input file, searches for the peaks in that spectrum, and then matches the peaks to reference values for the standard. A linear fit of pixel versus wavelength is performed, the parameters of which are outputted in the "coeffs.txt" file.

An example of running the software might be:

```
calibration.py peaks="<peaks_file>" input_file="input_file"
```

Where input spectrum will be a current spectrograph reading. The reference wavelengths file should include the wavelength values that peaks occur for the given standard, it can also intake a list formatted as "[values]". The output file should specify the name of the file that contains the fit parameters, the runtime, the date, the input spectrum, and the reference wavelengths, as well as some error analysis.

format-file The format-file.py script will take in a standard text file from the UV-Vis spectrophotometer, and format it in an input file for you. The parameters for this are input= and output=.

```
python3 format-file.py input="<uv-vis text file>" output="<name of output text file>"
```

Stated will be the inputted parameters and whether or not the file was written to.

Parameters:

```
{'input': 'abs_values.txt', 'output': 'in-out.txt'}
```

```
input = abs_values.txt
```

```
output = in-out.txt
```

Absorbance already present

Wavelength already present

2.1 Calibration

For running a calibration you input the absorbance, and the peaks. Peaks can either be defined as an text file with an array of values or a list:

```
peaks="[584.42, 685.26, 743.59, 807.87]"
```

```
peaks="./Peaks/reference_peaks.txt"
```

The default with no user input is the file:

```
./Peaks/reference_peaks.txt
```

The input file is specified as:

```
input_file="./Input-Files/in-out.txt"
```

The default with no user input is the file:

```
./Input-Files/in-out.txt
```

Include in the input file should be the absorbance values from the spectrophotometer.

Running the program with an input:

```
calibration.py peaks="./Peaks/reference_peaks.txt" input_file="./Input-Files/in-out.txt"
```

The program output in the terminal is:

```
Found 9 peaks in spectrum
```

```
There are 6 reference peaks
```

```
Found good fit with 6 peaks
```

```
Tested 84 combinations (15 valid)
```

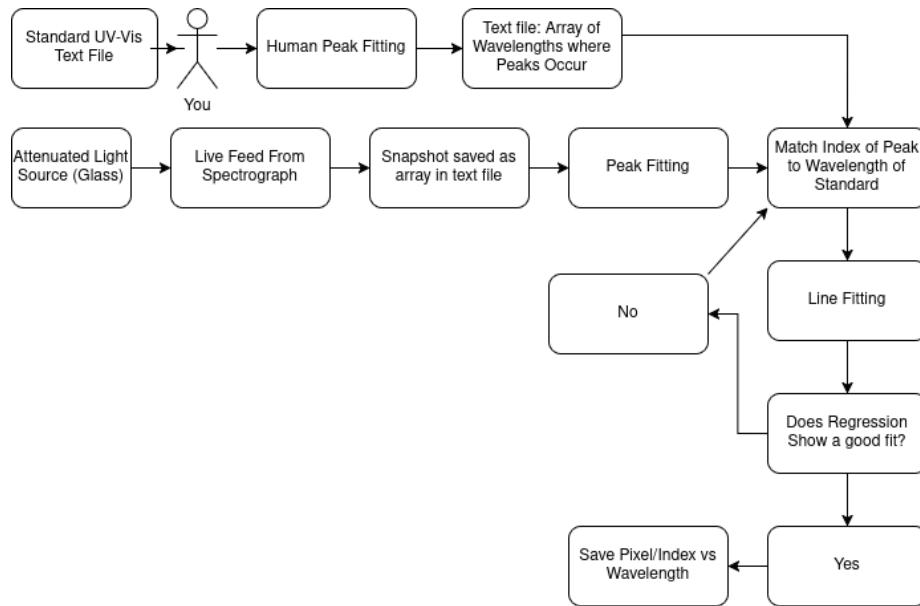
```
RMS: 0.0019949 nm, Max error: 0.00318477 nm
```

```
Runtime: 0.2147 seconds
```

The program outputs the number of fitted peaks, the number of peaks provided in your reference, the number of peaks used in the fit (as you can have

fewer reference peaks than fitted). Combinations provides the value of how many tested combinations there are, with a given number of valid fits. The lowest error fit is chosen. Written out in files are a "coeffs.txt" which provides the polynomial fit parameters, "./Figures" which provides plots of the fit, to give reference plots. Final-wise there is an "output.txt" that gives all input parameters, and output values.

The "coeffs.txt" is in an easy to read text value format for input into LAB-View.



3 Skeleton

Standardized Peaks

1. Standard UV-Vis Text file

- Timeline: Week 1
- Debugging: Checking Length of the File, if the length of the file is 0 or greater than 2,000,000 bytes flag.
- Standard UV-Vis Text file is saved using `numpy.savetxt`, in Peaks folder under "reference_peaks"

2. Manual Identification of Peaks

- You identify the peaks of the text file, record the wavelength of peaks, and store that as a text file with an array of wavelengths where the

peaks occurred.

- Timeline: Week 1
 - Debugging: Check if array in the text file is null. Check that array of peaks contains at least 3 peaks if not flag. Check that array of peaks contains less than 20 peaks if not flag Check to make sure the values are within a range 200-1000 (nm).
3. You identify the peaks of the text file, record the wavelength of peaks, and store that as a text file with an array of wavelengths where the peaks occurred.

Raw Data

1. Snapshot saved as an array in a text file
 - Working with the input text file from the spectrograph, in this regard I will have a mock text file that the normal output would appear as. This is because the text file comes from LABVIEW.
 - Timeline: Week 2
 - Debugging: Check if array in the text file is null. Check that array is within reasonable range of number of peaks. Check to make sure the values are within a range of pixels 200-2000.
2. Peak Fitting
 - Working with the input text file from the spectrograph, in this regard

I will have a mock text file that the normal output would appear as.

This is because the text file comes from LABVIEW.

- Timeline: Week 2-4 Finding a good peak finding algorithm
- Timeline: Week 3-5 Checking how the filtering affects the peak finding
- Timeline: Week 6 Tuning the peak fitting parameters.
- Debugging: Graphing to visually see where the peaks are fit. Giving an example text file with known peaks and making sure the output matches expected.

3. Match Index of Peak to Wavelength of Standard

- Using the peaks from the peak fitting match the pixel number to the wavelength from the known.
- The program will iteratively check every combination of reference to pixel peak.
- Timeline: Week 3-4
- Debugging: Graphing to visually see where the wavelengths are occurring. Checking that length of the output array is the same as the length of the input array. Compare Domain with a reasonable range, ie:0-2000 nm. Checking that the dispersion (nm/pixel) is within a reasonable range.

4. Line Fitting

- Fitting a line of best fit to the wavelength data to be able to interpolate the wavelengths between the peak points.
- Timeline: Week 4-5
- Debugging: Graphing to visually see the line of best fit. Reject if residuals are too high. Check that the coefficient values are too high for a cubic fit.

5. Save Pixel/Index output

- This will save out output line information and export it as a standardized text file. We'll start on this earlier than the rest because we'll need to be figuring out store text files and load them earlier.
- Timeline: Week 2-3, Week 4-6
- Debugging: Pulling up the array and pasting the output in the terminal. It's a short array so it shouldn't be hard to read the output manually. Checking the length of the text. Checking that it's not null. Graphing the fit. Checking that certain parameters, namely the slope is positive.

4 Bill of Materials

- **Python 3.14**
- matplotlib 3.10.8
- numpy 2.3.5
- scipy 1.17.0
- signal from scipy library
- itertools - combinations

5 Citations/Readings

Pan Du, Warren A. Kibbe, Simon M. Lin, Improved peak detection in mass spectrum by incorporating continuous wavelet transform-based pattern matching, *Bioinformatics*, Volume 22, Issue 17, September 2006, Pages 2059–2065, <https://doi.org/10.1093/bioinformatics/btl355>